

An Improved Exponent Bound for the Exact Discrete Fourier Transform

Research draft: round-six complex-network transfer

October 8, 2026

Under the exact-complex-arithmetic, specified-root, and logarithmic-word address model of OpenAI's *An explicit power saving for the exact discrete Fourier transform* [O], the proposed bound is

$$T(n) = O\left(n(\log n)^{1-\delta}\right), \quad \delta = \frac{73}{10^6} = 7.3 \times 10^{-5}.$$

This draft transfers the attributed round-six **complex** construction of Swapnil Jain [J], including prior whole-residual batching [B], two-stage topology [P], and copied-centre scheduling [C], into [O]'s uniform array model. It does not infer a Fourier bound from an integer-multiplication running time. The finite network is pinned to commit `f2176bc1124821bf17eb63725bd366d7bdc020a3` of `Swapnil-jain/integer-mult-kappa`.

Status. This is a proposed research result supported by a written transfer argument and reproducible exact finite checks. Neither this Fourier theorem nor the complete external construction has been independently reviewed or formally verified here. The external Lean file checks numerical certificates and their integer-multiplication assembly; it does not formalize the Fourier transfer below. No novelty or priority claim is made for the imported constructions.

The earlier three-stage result, with proposed saving `5.5e-10`, is preserved unchanged in `archive/three-stage/`. The separate short proof retains the simpler `5.2e-10` and `5.3e-10` fallbacks. Their proofs do not depend on this update.

1. The tensor bound sufficient for the Fourier transfer

Use the same fixed kernel as [O]:

$$C = \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}, \quad C^2 = X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad C^{-1} = XC.$$

The target intermediate result is an algorithm applying C on all k binary axes of an arbitrary complex array, in

$$O\left(2^k(k+1)^\theta\right), \quad \theta = 1 - a, \quad a = \frac{36926111}{500000000000}.$$

The cost must include scalar arithmetic, finite-table preparation, index preparation, array initialization, and data movement. The algorithm must work for every integer k , and with additional spectator axes. Section 8 proves this statement from the finite network and its strict moment certificate. Section 9 then substitutes it into [O], Proposition 4.2 and Section 5.4.

All finite network choices below are fixed independently of the requested Fourier length. Only rational constants and $\mathbf{i}$ are used by the new tensor routine. On variable data its operations are linear: addition, subtraction, and multiplication by prepared constants. The supplied root of order less than $1024 \cdot n^3$ from [O] remains sufficient. No bit-complexity, coefficient-size, or numerical-stability conclusion is asserted.

2. The pinned scalar producer

Fix $h = 24$ and put

$$\mathcal{T} = \binom{[h]}{3}, \quad v = 2024, \quad m = h^2 = 576, \quad N = v^2 = 4096576.$$

[J]’s **NStar3** producer is a finite addition DAG on inputs x_T , indexed by triples. It shares pair-exclusion sums and emits signed partial sums whose aggregate is

$$D_S - H_{2,S}, \quad D_S = \sum_{T \cap S = \emptyset} x_T, \quad H_{2,S} = \sum_{|T \cap S|=2} x_T.$$

It also retains the h totals

$$E_j = \sum_{T: j \notin T} x_T \quad (0 \leq j < h-1), \quad A = \sum_T x_T.$$

The omitted total is recovered by

$$E_{h-1} = (h-3)A - \sum_{j < h-1} E_j.$$

For a target triple S , define the retained-total readout

$$R_S = A - \frac{1}{2} \sum_{j \in S} E_j.$$

Its coefficient on x_T is $(|S \cap T| - 1)/2$. Therefore

$$R_S + \frac{1}{2} D_S - \frac{1}{2} H_{2,S} = x_S. \tag{2.1}$$

Indeed the four possible intersection sizes 0, 1, 2, 3 give coefficients 0, 0, 0, 1. The expression for the omitted E is used only in this source-dependent identity. No relation among arbitrary old scratch values is assumed.

For reproducibility, the finite object is specified by the **unmodified, hash-pinned** producer in `verification/vendor/jain_round6/producer.py`. The new checker rebuilds every support from the DAG's arguments and checks all $v^2 = 4,096,576$ entries of (2.1), including zero entries. It also checks every retained total. This is exhaustive finite arithmetic at the selected size, not a proof for every possible ground size.

The active producer has 28,944 addition nodes, 20,240 signed output uses, and 24 retained output uses. Its role compiler allocates one carrier per directed use and identifies one input carrier with the pivot output of each addition. Thus the number of physical auxiliary roles in one invocation is

$$R = 28944 + 20240 + 24 = 49208. \quad (2.2)$$

Every addition and fan-out is implemented by reversible additions into arbitrary existing values. Let L denote this invertible scratch mixer, V the source injection, and J the combined half-weighted piece and retained-total readout. Equation (2.1) is $JLV = I$. Starting with arbitrary scratch z , the chronological word

$$L, -J, L^{-1}, +V, L, +J, L^{-1}, -V \quad (2.3)$$

changes the target by $JL(z + Vx) - JLz = x$ and restores z . Here the signs on J and V specify additions to the target and scratch, respectively. Reversing the word and each gate implements the inverse shear. The scalar correctness of this construction holds on every scratch value.

3. Phase frames and the two stages

All label geometry is over the binary field; payloads remain complex. For a nondegenerate binary subspace U of the ambient m -dimensional space, use [O]'s frame

$$\Phi_U = H_m \operatorname{diag}_y \left(i^{\operatorname{wt}(P_U y)} \right) H_m,$$

where P_U is orthogonal projection and H_m is the normalized Walsh matrix. The Walsh matrices establish identities and are **not executed** by the algorithm. A scalar gate commutes with a common address frame on its touched roles. Consequently the inserted edge quotients telescope exactly as in [O], equation (2.6).

The local binary space is $D = \mathbb{F}_2^h$. Each source triple is labelled by its indicator line. For a producer node, use the span of its triple indicators when they share a common pair; otherwise use the coordinate space of their union. The common-pair vectors are orthonormal. These are [J]'s labels, with its output-piece splitting retained.

The following forward schedule implements (2.3). J_{ret} and J_{piece} are the two parts of J ; a source or target line is the line of that particular triple.

Step	Scalar operation	Local common label
1	Apply L	0
2	Subtract retained readout from target	0
3	Subtract piece readout from target	0
4	Apply L^{-1}	0
5	Add sources to their input carriers	Source line
6	Apply L	Producer-node label at each gate
7	Add retained readout to target	0, supplied by copies as in Section 4
8	Add piece readout to target	Target-line perpendicular space
9	Apply L^{-1}	D
10	Subtract sources from their input carriers	D

The inverse schedule reverses the operations, inverts their scalar gates, and complements every local label in D . It is not a reversal of the forward labels without complementation.

For a data pair (S, T) , write P and Q for its two indicator lines. Stage one performs the forward shear on the first triple index, for every fixed second index, with lifted label $U \otimes Q$. Stage two performs the inverse shear on the second triple index, with logical source and target exchanged, and with lifted local label

$$(P^\perp \otimes D) \perp (P \otimes U^\perp), \quad (3.1)$$

where U is the corresponding forward label before complementation. There are v invocations per stage, with **distinct** auxiliary banks for the two stages.

The input data frames are $\Phi_{P \otimes Q}$ on X and identity on Y . Their terminal frames are $\Phi_{D \otimes D}$ on X and $\Phi_{(P \otimes Q)^\perp}$ on Y . Each auxiliary has source frame identity and sink frame $\Phi_{D \otimes D}$. Stage-one auxiliaries pay an exit of rank $m - h$; stage-two auxiliaries pay an entrance of rank $m - h$. No translated auxiliary-source gauge is needed for this particular histogram.

The two interstage data residuals are both $P^\perp \otimes Q^\perp$, of rank $(h - 1)^2$. Each of the four remaining data-front edges has rank $h - 1$. A coordinate unit outside each triple supplies a norm-one vector in the corresponding complement; tensoring the bases preserves nondegeneracy and nonalternation.

For the local circuit, the verifier constructs symmetric binary projectors and replays every physical auxiliary edge in both orientations. It checks nesting and the existence of an orthonormal residual basis. It separately reproduces [J]’s 140,064 label checks with zero bad edges. Thus the new construction stays within [O], Lemma 2.2; it does not require the more general alternating-residual normal form used by some other follow-ups.

4. Copied retained centres, with their transformations charged

The retained slots are distinct terminal output-use carriers. After the middle producer, they are only read by the scatter before cleanup. Let U be one such carrier’s local label and $r = \dim U$.

In the forward invocation, its old path was

$$U \longrightarrow 0 \longrightarrow D,$$

at cost $r + h$. Copy the physical value at frame U , transform the copy to frame zero, supply all scatter reads from the copy, and discard it. The original remains at U until its direct move to D , of rank $h - r$. The copied transform is an inverse residual of rank r . The new cost is $r + (h - r) = h$, a saving of r .

In the reverse complementary invocation the old path is $0 \rightarrow D \rightarrow U^\perp$. Advance the original from zero to $U^\perp$, at rank $h - r$; copy it and transform the copy to D , at rank r , for the early readout. The original already has the frame required by the subsequent inverse mixer. In both orientations, the copy's transformed value is exactly the value the old readout would have seen. Its scalar reads do not modify it, and all later original incidences are preserved.

This is [C]'s copied-centre argument, specialized to [J]'s retained totals. There are $h - 1$ retained labels of dimension $h - 1$ and one of dimension h . The loss per local invocation is therefore

$$\ell = (h - 1)^2 + h = 553.$$

After the change, the internal residual histogram H_r has mass

$$\sum_r r H_r = h R + \ell. \quad (4.1)$$

Every copy transform remains in H_r . The original and copied values occupy full streams with all spectator coordinates. A fresh temporary need not be an additional arbitrary-input role: copying, reading and discarding it cost linear work, and its recursive transform is explicitly counted. One temporary can be reused by processing retained readouts sequentially. This is workspace, not an uncharged enlargement of the role count W .

5. The paid endpoint correction

Two shears give the scalar data map

$$(x, y) \mapsto (-y, x + y).$$

For one data pair let u generate $P \otimes Q$. It has binary norm one and integer weight nine. Set

$$T = \Phi_{\langle u \rangle}, \quad F = \Phi_{D \otimes D}, \quad E = \Phi_{\langle u \rangle^\perp} = F T^{-1}.$$

Starting with physical inputs x, y , the two-stage common-frame identity gives physical outputs

$$A = -Fy, \quad B = F T^{-2} x + Ey. \quad (5.1)$$

Apply T^{-1} to a **copy** of A , and add that result to B . Since $T^{-1}A = -Ey$, the corrected B is $F T^{-2} x$. This is one rank-one child per data pair, with copying, addition and erasure charged as linear work.

A final address translation suffices to normalize this endpoint in the array model. From the definition of the line frame,

$$T^2 = R_u, \quad R_u^2 = I,$$

where R_u translates binary addresses by u . All these operators commute. Thus $R_u B = Fx$ after the correction, while $-A = Fy$. Reordering the banks gives the desired F on both original data arrays. This is the same translation identity used in [O], Proposition 2.4, applied to the weight-nine line of the two-stage construction.

Every auxiliary scalar value is restored by its invocation. Its source/sink frame quotient is F , so it too receives exactly F , on arbitrary input data. The argument tensors over every column f and is unchanged by spectator bits. The correction has rank f after tensor lifting and is charged as a rank-one child on f active axes, once for each of the N data pairs.

6. Whole-residual recursive calls and the physical histogram

For a nested edge with orthonormal residual basis $z_1, \dots, z_r$, [O], Lemma 2.2 writes its operator as the product of r forward or inverse directional kernels. Use **one** binary basis extension placing these r independent directions in the first r coordinate slots. On f columns, collect their rf bits into one contiguous field.

In these coordinates all factors act on disjoint bits. The identity $C^{-1} = XC$ moves every inverse into a translation of its bit, so the entire residual is one $C^{\otimes rf}$ call with address permutations and translations before or after it. In particular, the directions' signs need not agree. This is whole-residual batching [B] specialized to the array primitives already allowed by [O]. The cost of the wrappers is established in Section 8, rather than assumed free.

The full list of child widths consists of the following disjoint classes:

Class	Number of children	Width
Internal auxiliary edges, including centre copies	$2vH_r$	r
Auxiliary exterior edges	$2vR$	$m - h = 552$
Interstage data edges	$2N$	$(h - 1)^2 = 529$
Data fronts	$4N$	$h - 1 = 23$
Endpoint copies	N	1

With

$$W = 2N + 2vR = 207387136, \quad L = 2v\ell = 2238544,$$

the sum of all child widths is

$$s = Wm - N + L = 119453132304, \quad Wm - s = 1858032. \quad (6.1)$$

Both orientations give the same H_r under the physical projector replay. The resulting full histogram is:

r	n_r	r	n_r
1	184811440	14	14864256
2	107199136	15	4849504
3	50583808	16	13014320
4	53478128	17	10196912
5	33031680	18	33371712
6	27611408	19	13601280
7	14767104	20	22774048
8	20195472	22	44038192
9	6217728	23	16479408
10	14119424	24	4048
11	5051904	529	8193152
12	13742960	552	199193984
13	4663296		

Unlisted widths have multiplicity zero. Every child has $1 \leq r \leq 552 < 576$, so there is no same-width recursion. The verifier checks the complete table against both a fresh run of [J]’s histogram builder and the pinned inputs to `Round6.lean`.

7. An exact rational moment certificate

Define

$$\Psi(\theta) = \frac{1}{W} \sum_r n_r \left(\frac{r}{m} \right)^\theta.$$

Use the rational saving actually checked by [J]’s Lean file,

$$a = \frac{36926111}{500000000000} = 0.000073852222, \quad \theta = 1 - a.$$

For rational upper bounds $U_r \geq \log(m/r)$,

$$\Psi(1 - a) = \sum_r \frac{r n_r}{W m} \exp\left(a \log \frac{m}{r}\right) \leq \sum_r \frac{r n_r}{W m (1 - a U_r)}, \quad (7.1)$$

provided $0 \leq a U_r < 1$. The inequality follows by comparing the exponential and geometric power series. The logarithm bounds use range reduction by powers of two and

$$\log y = 2 \sum_{j=0}^{K-1} \frac{z^{2j+1}}{2j+1} + \mathcal{R}_K, \quad z = \frac{y-1}{y+1}, \quad 0 \leq \mathcal{R}_K \leq \frac{2z^{2K+1}}{(2K+1)(1-z^2)}.$$

The new verifier uses the repository’s earlier logarithm implementation, rounded **upward** to a rational grid, rather than importing [J]’s logarithm routine. Exact arithmetic gives

$$\sum_r \frac{r n_r}{W m (1 - a U_r)} < 1 - \frac{7}{10^{14}} < 1. \quad (7.2)$$

Its actual rational gap is approximately $7.91950201573749 \times 10^{-14}$; the certificate stores the rational endpoints, not merely this decimal. This is a strict witness, not a claim to have attained the critical root of the moment equation. [J]'s tighter Python bound 7.3861113×10^{-5} is not needed here.

8. Uniform unequal-width recursion without role padding

Linear-time layout lemma. A fixed binary basis map on each of f columns, selection of any fixed r output slots per column, and fixed column translations can be implemented in $O(2^k)$ charged operations for $k = mf + b$, $0 \leq b < m$.

To see this, extend [O], Lemma 2.5 from one selected slot to r slots. Treat each m -bit column as a digit of fixed radix 2^m . Its finite table records contributions to the selected rf -bit field and to the spectator fields. Precompute their displaced contributions in $O(f + 1)$ time. Traverse the digit prefix tree, maintaining source and destination address sums. Every extension uses a fixed number of word operations; fewer than 2^{k+1} nodes are visited. At each leaf move the value once, using an output buffer. The inverse permutation reverses the source/destination addresses. Translations can be incorporated into the digit tables. This includes initialization, discovery and movement of the contiguous fibers and requires no per-entry scan of f columns.

Now let $T(k)$ be the cost of applying $C^{\otimes k}$ to W arbitrary arrays and put $t(k) = T(k)/(W2^k)$. Handle the fixed range $k < m$ directly. For $k \geq m$, set $f = \lfloor k/m \rfloor$ and process the fewer than m leftover axes directly. Apply the finite network on the remaining mf axes.

A child of width r acts on $F_r = 2^{k-rf}$ fibers of length 2^{rf} in **one** original or temporary stream. Pack complete groups of W fibers and run this same simultaneous algorithm recursively. Process the fewer than W leftover fibers by the ordinary coordinate algorithm. This gives

$$\begin{aligned} T(k) &\leq AW2^k + \sum_r n_r \left\lfloor \frac{2^{k-rf}}{W} \right\rfloor T(rf) \\ &\quad + B(W-1) \sum_r n_r rf 2^{rf}. \end{aligned} \tag{8.1}$$

The first term charges the fixed number of scalar gates, all layout passes, temporary copies, endpoint translations, and leftover axes. All constants are independent of k . Since $r < m$ and $k \geq mf$,

$$rf 2^{rf-k} \leq rf 2^{-(m-r)f} \leq r.$$

Consequently incomplete batches contribute only a bounded additive term after normalization:

$$t(k) \leq A' + \frac{1}{W} \sum_r n_r t(r \lfloor k/m \rfloor). \tag{8.2}$$

Choose a constant K large enough for the finite base range and for $K(1 - \Psi(\theta))m^\theta \geq A'$. Strong induction on k , using $rf \leq (r/m)k < k$, gives

$$t(k) \leq A' + Kk^\theta \Psi(\theta) \leq Kk^\theta \quad (k \geq m).$$

Thus $T(k) = O(W2^k(k+1)^\theta)$. A single requested transform initializes $W - 1$ other arrays once and retains its own output; because W is fixed, this adds only $O(2^k)$ work.

The recursion can be performed serially with reusable buffers and temporary streams. Each child array has at most half as many entries as its parent, since $k - rf \geq 1$. The maximum axis count also contracts by at most 552/576, giving $O(\log(k+2))$ stack depth. Summing live array sizes along the stack gives $O(W2^k)$ workspace. Addresses, counters and strides fit a fixed number of $O(k+1)$ -bit words. Fixed tables and scalar coefficients have constructive finite preparation procedures, so their initial construction is charged as a constant. These facts establish the uniform arbitrary-input tensor interface required by [O].

9. Propagation to every Fourier length

The proof of [O], Proposition 4.2 uses its tensor routine only through the bound $O(2^k(k+1)^\theta)$ on arbitrary complex arrays, including index and movement costs. Replacing that routine by Section 8 therefore gives

$$O\left(R(\ell+1)^\theta(1+\log r_{\max})^4\right) + \text{poly}(\ell, r_{\max}, \log(R+2)).$$

The exact-width local Fourier compiler, its scalar preparation and denominator conditions are unchanged. Its sectors still carry the same kernel C . Applying [O]’s small-prime working lengths, CRT permutations, and charged chirp convolution, with $\ell = \Theta(\log n / \log \log n)$, gives

$$T(n) = O\left(n(\log n)^\theta(\log \log n)^{4-\theta}\right). \quad (9.1)$$

In particular, the transform of the fixed convolution operand remains charged. The specified root order and logarithmic-word address bounds remain those of [O].

Finally,

$$\delta = \frac{73}{10^6}, \quad a - \delta = \frac{426111}{500000000000} > 0.$$

For every fixed positive exponent gap, the fixed power of $\log \log n$ in (9.1) is eventually smaller than $(\log n)^{a-\delta}$. This proves the proposed pure-power corollary, conditional on the specified finite construction and upstream results. There is no integer-multiplication bit-network constraint or extra factor-of-two loss in this transfer.

The new proposed **delta** is about 132,727 times the earlier 5.5e-10 value and 730,000,000 times [O]’s published 1e-13 headline. These ratios compare asymptotic exponent savings, not running times or practical FFT performance. The previous centre reduction and shared-sum gains are not added to the imported construction’s saving.

10. Reproduction and proof boundaries

Run from the repository root with standard-library Python 3.10 or newer:

```
python3 verification/run_checks.py
```

For the new component alone:

`python3 verification/verify_round6.py`

The new verifier checks source hashes; reconstructs all producer supports and the full local coefficient matrix; reproduces the external binary label checks; replays physical auxiliary paths in both orientations using the earlier repository’s binary-projector implementation; charges centre copies; rebuilds and compares the entire histogram; checks dirty-scratch scalar trials and exact endpoint identities; and certifies the rational moment and absorption slack. Negative controls reject unpaid centre copies, an omitted endpoint correction, and an excessive exponent saving. The report is compared with `verification/certificates/round6_reference.json`.

The producer and the external frame/histogram checks are vendored source, not independent implementations. The extra replay and coefficient checker were written with the same assistant and reuse earlier local binary algebra. The dirty-scratch executions are finite exact rational trials; the universal scratch statement uses (2.3) and the copied-read proof. Endpoint tests use exact Gaussian-integer algebra. The checker does not execute the astronomical full Fourier algorithm, prove the infinite recurrence by enumeration, or run Lean. The pinned Lean source and numerical inputs are included for comparison, with their original license and notice.

Independent mathematical review should prioritize the copied-read timing, the lifted data-frame interfaces, the paid endpoint correction, the full-residual layout lemma, and the remainder recurrence. The original compiler and all-length reduction are invoked with their stated hypotheses, not re-proved here. The earlier result and its verification suites remain separately available.

References and attribution

[O] OpenAI. *An explicit power saving for the exact discrete Fourier transform*, September 25, 2026. Especially Lemmas 2.2 and 2.5, Proposition 2.4, Theorem 2.6, Proposition 4.2, and Section 5. [Public manuscript](#).

[J] Swapnil Jain. `integer-mult-kappa`, round-six complex producer and certificates, pinned at `f2176bc1124821bf17eb63725bd366d7bdc020a3`. [Complex implementation](#), [Lean numerical certificate](#). The source attributes research and implementation assistance to Claude (Anthropic); its Apache-2.0 license and full NOTICE are preserved locally.

[B] icekylinx. Whole-residual batching and unequal-width recursion, PR #10 of Colkitt’s repository, commit `62691e3`; further full complex batching is credited to eumemic, PR #15. [Batching proof](#), PR #15.

[C] icekylinx. Copied retained centres and two-stage complex endpoint transfer, PR #36, commit `11817ccacb564bb7f98789c20dc11d3fece207e3`. [Copied-centre lemma](#), [complex endpoint argument](#). The source records substantial GPT-6 Astra and Codex assistance.

[P] Aurel Prosz (Paurel). Two-stage topology and charged endpoint-copy correction, commit `c82d09eb4781b68435e3fa3eb6beea72fa900fab` of [integer-mult-bounds](#), as attributed in [J] and [C].

The common finite-network framework originates in OpenAI’s *Integer multiplication below $n \log n$* (#109); Douglas Colkitt’s `CrocSwap/integer-mult-bounds` supplies the follow-up framework. Other antecedents, including contributions by Zhihao Chen / jacklightChen and retained-total and complex-circuit work, are identified in the preserved external NOTICE. This draft claims a proposed transfer to #130’s arithmetic model, not authorship of those network improvements. AI-assisted checking is not independent review.